MEMORY COMPILER

Agent-Ready Technical Specification

A multimodal deep-learning system for natural-language search across long-form video archives

PRIMARY BUILD GOAL

Implement a reliable end-to-end vertical slice: upload a video, index visual frames, search in natural language, and open the original video at the matching timestamp. Then add temporal/action retrieval in measured, testable increments.

Document purpose

This document is intended to be handed to an autonomous coding agent (such as Codex or Antigravity). It defines the product scope, technical architecture, interfaces, data contracts, implementation order, test strategy, acceptance criteria, and guardrails. It is a build specification, not a claim that every proposed model or advanced capability already works.

Project principles

- Build a working vertical slice before adding multiple expensive models.
- Use pretrained encoders first; train or fine-tune only after a baseline and evaluation set exist.
- Treat generated captions and action labels as hypotheses supported by video evidence, not guaranteed facts.
- Measure retrieval quality, temporal localization, processing cost, and latency.
- Keep models behind interfaces so model choices can change without rewriting the application.

Target audience: developer / coding agents implementing the repository and human reviewers validating the result.

Contents and how to use this specification

Section	Topic
01	Product definition and scope
02	Requirements and acceptance criteria
03	Architecture and technology decisions
04	Data model and persistence
05	Video ingestion and indexing pipeline
06	Search and temporal reasoning
07	API contracts
08	Frontend experience
09	Repository structure and configuration
10	Implementation phases and definition of done
11	Testing and evaluation
12	Privacy, security, reliability, and operations
13	Agent execution protocol
14	Risks, open decisions, and non-goals
15	Reference links and terminology

RECOMMENDED EXECUTION ORDER

Implement Phases 0–3 first. Do not begin temporal event reasoning, person tracking, or fine-tuning until upload, processing status, frame indexing, semantic search, timestamp playback, and tests work end to end.

Agent usage: give the entire document to the coding agent. Ask it to inspect the existing repository first, summarize the current state, then implement one phase at a time with tests. If the repository already contains working code, preserve it and adapt the plan rather than replacing it blindly.

1. Product definition and scope

1.1 Product summary

Memory Compiler is a private video archive with natural-language retrieval. A user uploads videos and asks for moments such as “find the person in a red shirt entering the room” or “find a moment when someone hands an object to another person.” The system returns candidate timestamps, supporting thumbnails, short descriptions, and playback links into the source video.

The product should be understood as a retrieval system with evidence and temporal metadata—not as a general chatbot, an autonomous surveillance platform, or a guarantee that every action in every video can be recognized.

1.2 Primary users and user stories

- As a user, I can upload a video and see whether it is queued, processing, ready, or failed.
- As a user, I can search my indexed videos using natural language.
- As a user, I can see ranked results with the source video, timestamp range, thumbnail, and explanation of the match.
- As a user, I can click a result and play the original video from the matching timestamp.
- As a user, I can delete a video and its derived index data.
- As a developer, I can inspect processing errors and reproduce model/indexing settings for a video.

1.3 MVP scope (must ship first)

- Single-user local development mode, with ownership-aware data structures ready for multi-user extension.
- Video upload with validation, metadata extraction, and asynchronous processing status.
- FFmpeg-based frame extraction with timestamps and configurable sampling interval.
- Pretrained image-text embeddings for sampled frames.
- PostgreSQL + pgvector for frame metadata and vector search.
- Natural-language query embedding and top-K visual result retrieval.
- React UI for library, processing status, search results, and timestamp-based playback.
- Unit and integration tests for core paths; deterministic mock model for CI tests.

1.4 Post-MVP scope (do not block MVP on these)

- Video-window embeddings and action-aware retrieval.
- Audio transcription and audio event retrieval.
- Object detection, tracking, clothing attributes, and participant-role matching.
- Generated event descriptions and an event graph.
- Temporal query parsing for before/after/first/last and multi-event constraints.
- Fine-tuning or contrastive learning for temporal-aware retrieval.
- Multi-user production deployment, cloud object storage, and advanced scaling.

1.5 Explicit non-goals

- Training a foundation video model from scratch.
- Guaranteeing correct identification of every person, action, object, or intention.
- Cross-video real-world identity recognition or facial recognition.
- Automatic legal, safety, disciplinary, or employment decisions based on retrieved video.
- A full video editor, generative video tool, or general-purpose conversational assistant.

2. Requirements and acceptance criteria

Use these requirement IDs in issues, pull requests, and agent status updates. “Must” means required for MVP; “Should” is a planned enhancement; “Later” is explicitly deferred.

ID	Priority	Requirement	Acceptance evidence
FR-01	Must	Upload supported video files and return a video ID without waiting for full indexing.	API returns 202 or equivalent queued response; DB row exists.

ID	Priority	Requirement	Acceptance evidence
FR-02	Must	Expose processing state and stage.	Status endpoint returns queued/processing/ready/failed plus safe error summary.
FR-03	Must	Extract metadata and timestamped frames.	Unit/integration tests verify duration, frames, and monotonic timestamps.
FR-04	Must	Generate and persist one visual embedding per sampled frame.	Embedding dimension/model ID stored and validated.
FR-05	Must	Search indexed frames from natural-language text.	Known fixture queries retrieve expected clip within configured top-K.
FR-06	Must	Return result timestamp, source video, thumbnail/evidence, and similarity score.	Response schema validated; score is not mislabeled as probability.
FR-07	Must	Seek original playback to result timestamp.	Frontend test verifies player time updates and begins playback when allowed.
FR-08	Must	Delete a video and its derived records/files.	No orphan records or files for the deleted video after successful deletion.
NFR-01	Must	Core processing is idempotent or restart-safe.	Retrying a failed job does not create duplicate frame/event rows.
NFR-02	Must	API and worker report actionable errors.	Logs include job/video IDs and safe failure stage; no silent failure.
NFR-03	Must	Configuration is externalized.	Model, frame interval, file size limit, storage path, DB URL, and worker settings use env/config.
NFR-04	Must	Protect private uploaded media.	Only authorized owner can list/search/play/delete their media; files not publicly exposed by default.
NFR-05	Must	CI does not require a GPU or download large model weights.	Mock/test backend runs all core tests without a GPU.
ML-01	Should	Support video-window embeddings.	Feature flag/model adapter enables video encoder path.
ML-02	Should	Represent actions and temporal relations.	Structured event/query schemas and temporal constraint tests exist.
ML-03	Later	Fine-tune retrieval on annotated hard negatives.	Only begins after baseline benchmark and reproducible evaluation exist.

3. Architecture and technology decisions

3.1 Logical architecture

Architecture overview (logical, not a deployment mandate)



For the MVP, API and worker may run on the same development machine, but long-running inference must execute outside the request handler. FastAPI BackgroundTasks is suitable only for small tasks; use a queue/worker for CPU/GPU-heavy processing.

3.2 Recommended stack

Layer	Recommendation	Reason / boundaries
Frontend	React + Vite + TypeScript; Tailwind CSS optional	Library, upload state, natural-language search, result playback.
API	Python + FastAPI + Pydantic	Validation, auth/ownership boundary, upload, status, search, deletion.
Worker	Celery + Redis for long-running processing	Retries, independent processes, queue state, GPU worker separation.
Media	FFmpeg/FFprobe; OpenCV as needed	Metadata, frame decoding, thumbnails, audio extraction.
Scene detection	PySceneDetect, optional in MVP	Candidate scene boundaries; not a substitute for event detection.
ML runtime	PyTorch; Hugging Face Transformers or OpenCLIP adapter	Pretrained encoder inference, later video model / fine-tuning.
Database	PostgreSQL + pgvector	Relational metadata and vector search in one store.
Object storage	Local private directory for MVP; S3-compatible storage later	Original videos and thumbnails; never store large binary files in DB rows.
Testing	pytest; frontend test tooling chosen in repo	Unit, integration, API contract, and smoke tests.
Packaging	Docker Compose for local dependencies	Reproducible PostgreSQL/Redis/worker/API development environment.

3.3 Architecture rules

- Use interfaces/adapters for embedding models, video encoders, detectors, transcribers, and storage. Do not couple business logic to one model library.
- Store model name, model version/revision, embedding dimension, preprocessing version, and indexing config with derived records or job metadata.
- Keep raw video timestamps as seconds from the original media timeline. Never infer timestamps solely from frame number.
- Use cosine similarity only with compatible embeddings and preprocessing. Do not compare vectors from different embedding spaces.

- Start with exact vector search if the dataset is small. Add approximate indexes only after measuring recall and latency.
- Do not call a generic LLM for every frame. Batch embeddings and reserve expensive interpretation for top candidates or explicit processing stages.

3.4 Model selection policy

The agent may select a suitable pretrained image-text model (for example, an OpenCLIP implementation) for the first baseline, but must record the exact model ID, weight license, expected embedding dimension, device behavior, and preprocessing. Do not assume all CLIP-family checkpoints or video models have identical licenses, APIs, context windows, or capabilities.

Make model selection configurable and provide a deterministic mock embedding provider for tests. The app must fail with an actionable message if the configured model cannot load; it must not silently switch to random vectors or a different embedding model in production.

4. Data model and persistence

Use migrations for schema changes. UUIDs are recommended for public-facing IDs. Add created_at/updated_at timestamps and appropriate indexes. Enforce ownership through query filters and database relationships, not only frontend checks.

4.1 Core entities

Entity	Suggested columns (not a final migration)	Purpose
videos	id, owner_id, filename, storage_key, mime_type, size_bytes, duration_seconds, width, height, fps, has_audio, status, current_stage, progress, error_code, error_message, created_at, updated_at	Original video and processing state.
processing_jobs	id, video_id, status, attempt, stage, progress, config_json, started_at, finished_at, error_summary, created_at	Restartable job tracking; make retries idempotent.
scenes	id, video_id, start_seconds, end_seconds, thumbnail_key, caption, detector_version	Optional scene units. A scene is not guaranteed to be an event.
frames	id, video_id, scene_id nullable, timestamp_seconds, image_key, width, height, embedding vector(N), embedding_model, preprocessing_version, created_at	Timestamped sampled frames and visual vectors.
video_windows	id, video_id, start_seconds, end_seconds, embedding vector(M), model_id, created_at	Phase 2: temporally encoded clips/windows.
events	id, video_id, start_seconds, end_seconds, description, action_label nullable, confidence nullable, provenance_json, embedding vector(K) nullable, model_id, created_at	Phase 3+: candidate event descriptions with evidence provenance.
tracks	id, video_id, scene_id, local_track_key, start_seconds, end_seconds, attributes_json, tracker_version	Phase 3+: local track identifiers; do not imply persistent real-world identity.
event_participants	event_id, track_id, role, confidence nullable	Phase 3+: roles such as actor/giver/receiver.
search_feedback	id, owner_id, query_text, result_id nullable, feedback_type, created_at	Optional user feedback for evaluation; do not auto-retrain from it.

4.2 Database invariants

- Foreign keys must cascade or trigger explicit cleanup consistent with the deletion policy.
- Add unique constraints for deterministic extraction identity, e.g. (video_id, timestamp_seconds, preprocessing_version) or an extraction key.
- Store the embedding model/version on every vector-bearing dataset; do not mix vector spaces in one search.
- Use database transactions for metadata state changes. Long-running model inference must not run inside an open DB transaction.

- Add indexes for video_id, owner_id, status, and timestamp fields used in filters and ordering.
- If pgvector dimensions are fixed in a column, choose the dimension from the selected model and document migration implications if model changes.

4.3 Example event record

```
{
  "id": "event_009",
  "video_id": "video_001",
  "start_seconds": 42.2,
  "end_seconds": 45.7,
  "description": "A person appears to pass a microphone to another person.",
  "action_label": "object_transfer",
  "provenance": {
    "source": "video_caption_model",
    "model_id": "configured-model-id",
    "evidence_frame_ids": ["frame_021", "frame_022", "frame_023"],
    "status": "unverified_model_hypothesis"
  }
}
```

The example deliberately avoids asserting clothing colors or participant roles unless those properties have been observed and validated. Store uncertain interpretations as hypotheses, with evidence links and provenance.

5. Video ingestion and indexing pipeline

5.1 End-to-end state machine

```
UPLOADED -> QUEUED -> PROBING -> EXTRACTING_FRAMES -> EMBEDDING_FRAMES
      -> PERSISTING_INDEX -> READY

Any active state may transition to FAILED with a safe error code.
A retry returns FAILED -> QUEUED and resumes/rebuilds idempotently.
```

5.2 Processing steps

Step	Implementation requirements
1. Validate input	Allow configured formats only. Enforce file-size limit, safe filename handling, and a generated storage key. Do not trust client MIME type alone.
2. Persist original	Write to private storage atomically. Create video and job records. Return a job/video ID promptly.
3. Probe media	Use FFprobe to get duration, dimensions, frame rate, codec, and audio-stream presence. Reject malformed or unsupported media with a safe status.
4. Optional scene detection	Use PySceneDetect for candidate boundaries. Treat detection as metadata, not event labels.
5. Extract frames	Use configurable interval and additional frames around scene boundaries. Store original timestamps and deterministic extraction IDs.
6. Preprocess frames	Resize/normalize according to the selected encoder; generate thumbnails separately if display size differs from model input.
7. Generate embeddings	Batch frames through the selected pretrained image-text encoder. Check dimension, finiteness, and expected normalization.
8. Persist index	Write frame rows/embeddings in batches. Update progress at reliable checkpoints.

Step	Implementation requirements
9. Finalize	Mark job READY only after all required artifacts are persisted and searchable. Record runtime and config/model versions.
10. Cleanup/error handling	On failure, retain enough diagnostic metadata to retry. Do not leave video marked ready with a partial index.

5.3 Sampling strategy

Default MVP strategy: configurable fixed-interval sampling (start with 2 seconds as a tunable heuristic), plus extra sampling around scene boundaries if scene detection is enabled. Do not hard-code the interval in multiple modules. Store it in the indexing configuration. Brief actions may be missed; this limitation must be documented and measured.

Phase 2 can add adaptive sampling. For example, detect high-motion intervals or use candidate search results to trigger denser indexing/re-analysis. Adaptive sampling must be evaluated against fixed sampling for both retrieval and cost.

5.4 Resource and failure handling

- Process in bounded batches so a long video cannot consume unbounded RAM or GPU memory.
- Use timeouts and subprocess return-code checks for FFmpeg/FFprobe operations.
- Make processing idempotent: retries should upsert or clean-and-rebuild derived rows rather than duplicate them.
- Store stage progress that can be estimated honestly; do not report invented progress percentages.
- Capture structured logs with job_id, video_id, stage, elapsed time, and a safe error code. Never log raw video frames or private media content.
- Separate CPU preprocessing from GPU inference if the deployment requires it. Keep the design able to run CPU-only tests.

6. Search and temporal reasoning

6.1 MVP retrieval: frame-level semantic search

1. Validate query length and search scope (owned videos only).
2. Encode the query using the exact compatible text encoder for the indexed visual model.
3. Query pgvector (or a well-encapsulated FAISS adapter) for top candidate frame embeddings.
4. Merge near-duplicate candidates from the same video into timestamp windows, preserving the best supporting frame.
5. Optionally diversify results so one dense scene does not fill every top-K slot.
6. Return ranked candidates, timestamp, thumbnail/evidence frame, similarity score, model/index version, and video link metadata.

> **MVP LIMITATION**
>
> Frame-level semantic search can locate objects and scenes, but it cannot reliably prove an action such as “hands a phone to another person.” Label this version as semantic frame retrieval. Do not market it as full temporal reasoning until Phase 4 is implemented and evaluated.

6.2 Candidate-to-fine retrieval (Phase 2+)

```
Natural-language query
  -> query embedding
  -> frame/window/event candidate retrieval (top K)
  -> merge overlapping candidate intervals
  -> optional video-model / caption-model verification
  -> temporal refinement
  -> structured constraints (roles, before/after, first/last)
  -> ranked results with evidence and uncertainty
```

6.3 Structured query schema

The first parser should return a validated, limited schema. If parsing fails or the query is outside the supported set, fall back to plain semantic retrieval and mark that temporal constraints were not applied.

```
{
  "raw_query": "Find a person picking up a phone before sitting down",
  "entities": [{"type": "object", "label": "phone"}],
  "events": [
    {"label": "pick_up", "entity_refs": ["person_1", "phone_1"]},
    {"label": "sit_down", "entity_refs": ["person_1"]}
  ],
  "temporal_constraints": [
    {"left_event": 0, "relation": "before", "right_event": 1}
  ],
  "ranking_mode": "relevance"
}
```

This JSON is a target shape, not a mandate to use an LLM for parsing. Start with a constrained parser or schema-validated model response. Do not execute arbitrary SQL generated by a language model.

6.4 Retrieval scoring and confidence

Keep ranking components separately observable: frame similarity, window similarity, event-description similarity, constraint satisfaction, and optional verification score. If a combined score is introduced, document normalization and weights in configuration and evaluate the effect of each component. A similarity score is not a probability unless a separate calibration procedure validates that interpretation.

6.5 Temporal constraints

Constraint	Initial interpretation	Validation rule
before	Event A must end before Event B begins.	Reject or penalize candidates that violate the interval order.
after	Event A must begin after Event B ends.	Equivalent inverse relation with explicit ordering.
during	Event A occurs within Event B interval.	Check interval containment within configured tolerance.
first / last	Return earliest/latest supported qualifying interval.	Apply only after candidate relevance and query constraints are assessed.
same person	Two events appear associated with the same local track.	Only claim within track scope; do not infer across videos or scene cuts.
near	Events occur within a configurable time tolerance.	Use a declared tolerance; do not silently choose arbitrary proximity.

6.6 No-match and uncertainty behavior

- Support an empty result when no candidate passes configurable relevance criteria.
- Distinguish “no matching result found” from “video indexing failed” and “temporal constraint unsupported.”
- Show the basis of a match (e.g. similarity retrieval, event-caption match, or verified temporal relation).
- Avoid definitive prose when only a generated description supports the event. Use cautious phrasing such as “possible match” and expose evidence frames.

7. API contracts

Use versioned routes under /api/v1. Define Pydantic request/response models. Return consistent error bodies. Include OpenAPI examples and validate responses in tests.

7.1 Endpoints

Method	Route	Input	Success	Behavior
POST	/api/v1/videos	multipart upload	202 Accepted	Create video record and enqueue processing; return video_id and status.
GET	/api/v1/videos	query: cursor/limit	200 OK	List only videos owned by the current user.
GET	/api/v1/videos/{video_id}	path ID	200 OK	Return metadata and processing status.
GET	/api/v1/videos/{video_id}/status	path ID	200 OK	Return current stage, progress, and safe error summary.
POST	/api/v1/search	JSON query	200 OK	Search only authorized/indexed videos; return ranked timestamped results.
GET	/api/v1/videos/{video_id}/stream	path ID + Range	200/206	Stream private original media with HTTP Range support; enforce ownership.
DELETE	/api/v1/videos/{video_id}	path ID	204 No Content	Delete video, derived rows, thumbnails, and stored file per policy.
GET	/health/live	none	200 OK	Process liveness only.
GET	/health/ready	none	200/503	Dependencies and readiness suitable for deployment health checks.

7.2 Upload response example

```
{
  "video_id": "uuid",
  "status": "queued",
  "status_url": "/api/v1/videos/uuid/status"
}
```

7.3 Search request example

```
{
  "query": "Find a blue car beside a building",
  "video_ids": ["uuid-1"],
  "limit": 10,
  "include_evidence": true
}
```

7.4 Search response example

```
{
  "query": "Find a blue car beside a building",
  "search_mode": "frame_semantic_v1",
  "results": [
    {
      "video_id": "uuid-1",
      "filename": "festival.mp4",
      "start_seconds": 83.0,
      "end_seconds": 87.0,
      "evidence_timestamp_seconds": 84.0,
      "thumbnail_url": "/api/v1/frames/frame-id/thumbnail",
      "description": "Visually similar frame candidate",
      "score": 0.82,
      "score_type": "cosine_similarity",
      "applied_constraints": [],
      "evidence": [{"frame_id": "frame-id", "timestamp_seconds": 84.0}]
    }
  ]
}
```

Numeric scores above are illustrative only. Do not hard-code these values or label them confidence percentages. The API should clearly state which retrieval mode produced the result.

7.5 Error response shape

```
{
  "error": {
    "code": "VIDEO_UNSUPPORTED_FORMAT",
    "message": "This video format is not supported.",
    "request_id": "request-id"
  }
}
```

Never expose stack traces, filesystem paths, credentials, raw exception dumps, or other users' metadata in API responses. Detailed diagnostic traces belong in protected server logs.

8. Frontend experience

8.1 Required screens

Screen	Required behavior	States to handle
Video library	List uploaded videos with thumbnail, duration, status, and actions.	Empty state, loading, failed processing, pagination.
Upload	Select or drag/drop a video; show validation errors and queued response.	Oversize, unsupported format, network error, duplicate submission.
Processing detail	Show honest pipeline stage and progress; poll or refresh status.	Queued, processing, ready, failed, retryable.
Search	Natural-language query, optional selected videos, result limit.	Empty query, unsupported query details, no matches, API error.
Results + player	Ranked results, evidence thumbnail, time range, description, play-at-time action.	Missing thumbnail, inaccessible video, playback seek failure.
Delete confirmation	Explain that the source and derived data will be deleted.	Cancel, failed deletion, success refresh.

8.2 UX rules

- Do not display a result until the related video is indexed and accessible to the current user.
- Display times consistently as HH:MM:SS or a concise time format, but send seconds in API data.
- Use the result start_seconds to seek the original video; do not create a separate clip unless explicitly required.
- Make uncertainty visible and avoid saying an action definitely occurred when only semantic similarity supports it.
- Show progress as stage-based unless the worker can report reliable numeric progress.
- Support keyboard access, descriptive labels, clear loading states, and usable empty/error states.

9. Repository structure and configuration

9.1 Suggested monorepo

```
memory-compiler/
  README.md
  .env.example
  docker-compose.yml
  docs/
    architecture.md
    api.md
    evaluation.md
  backend/
    pyproject.toml
    app/
      main.py
      core/           # settings, logging, errors
      api/v1/         # route handlers
      schemas/        # Pydantic request/response models
      db/             # session, models, migrations
      services/       # upload, search, deletion, ownership
      workers/        # queued processing jobs
      pipeline/       # probe, frame extraction, embedding index
      ml/             # model interfaces and adapters
      storage/        # local/object storage abstraction
    tests/
      unit/
      integration/
      fixtures/
  frontend/
    package.json
    src/
      api/
      components/
      pages/
      hooks/
      types/
  scripts/
    seed_demo_data.py
    evaluate_retrieval.py
```

9.2 Configuration requirements

Variable	Example/default	Rule
APP_ENV	development	Controls dev/test/prod behavior.
DATABASE_URL	required	PostgreSQL connection string; never commit credentials.
REDIS_URL	required if Celery enabled	Queue/broker URL.
MEDIA_ROOT	./data/media	Private local storage directory for development.

Variable	Example/default	Rule
MAX_UPLOAD_MB	configurable	Maximum accepted upload size.
ALLOWED_VIDEO_EXTENSIONS	.mp4,.mov,.mkv (example)	Validate actual media with FFprobe too.
FRAME_SAMPLE_SECONDS	2.0 (initial heuristic)	Single source of truth for fixed sampling interval.
EMBEDDING_PROVIDER	mock in tests; real provider in dev/prod	Select provider adapter explicitly.
EMBEDDING_MODEL_ID	required for real provider	Exact checkpoint/revision identifier.
EMBEDDING_DEVICE	auto/cpu/cuda	Device selection with CPU fallback only if deliberately configured.
EMBEDDING_BATCH_SIZE	configurable	Avoid GPU/CPU memory exhaustion.
VECTOR_DIMENSION	from model config	Validate against DB schema and generated outputs.
LOG_LEVEL	INFO	Structured logs; avoid media payloads and secrets.
CORS_ORIGINS	local UI URL in dev	Explicit allowlist; do not use wildcard credentials in production.

9.3 Developer experience

- Provide one documented command path to start local dependencies, backend API, worker, and frontend.
- Provide database migrations and an explicit seed/demo command; no invisible DB schema creation in production startup.
- Provide a health endpoint and a small smoke-test script.
- Provide a CPU-safe mock mode so the app and tests can be developed before downloading model weights.
- Document model download sizes and licenses before pulling heavyweight dependencies.

10. Implementation phases and definition of done

Agents should implement phases in order. Each phase ends with a working state and a testable acceptance gate. If scope or time is limited, prefer a complete earlier phase over partially implemented later features.

Phase	Tasks	Exit criteria
Phase 0 — Repository assessment and scaffolding	Inspect existing files, identify what already works, preserve viable code, write a short implementation plan, establish formatting/lint/test commands, and create env example + local dependencies.	Repository inventory documented; app skeleton starts; no secrets committed; smoke test succeeds.
Phase 1 — Upload and job state	Implement video upload, private storage, metadata record, queue/job abstraction, and status endpoint. Start with safe local storage.	Upload returns video ID promptly; status transitions queued/processing/ready or failed; validation and ownership tests pass.
Phase 2 — Frames and metadata	Implement FFprobe, timestamped frame extraction, optional scene detection, deterministic filenames, and retry-safe persistence.	Tests use a short fixture video; timestamps are monotonic; reruns do not duplicate records.
Phase 3 — Frame embedding search	Implement a model adapter, batching, pgvector storage, query text embedding, top-K search, result merge, and source timestamp playback. Keep mock provider for CI.	A known query against demo clips retrieves a relevant frame in top-K; UI jumps to correct timestamp; CPU CI does not download weights.
Phase 4 — Evaluation baseline	Create a small labeled benchmark; define query categories and expected intervals; report Recall@K, tIoU where applicable, search latency, indexing time.	Repeatable evaluation script outputs metrics and model/config metadata; baseline results are saved.
Phase 5 — Video-window retrieval	Add video encoder adapter, overlapping windows, vector records, candidate merging, and baseline comparison.	Frame-only vs video-window results are compared on the same benchmark; no unsupported claim that one is always better.

Phase	Tasks	Exit criteria
Phase 6 — Event descriptions and temporal constraints	Add candidate captions/events, provenance, constrained query schema, before/after/first/last evaluation, and explicit unsupported-query fallback.	Unit tests cover order reversals, false event matches, missing events, and unsupported constraints.
Phase 7 — Hardening and demo	Add robust deletion, range streaming, security checks, UI polish, logging, deployment instructions, and failure-mode tests.	Fresh checkout setup follows README; end-to-end smoke test passes; no cross-user media access; deletion cleans derived assets.

10.1 Suggested schedule (not a promise)

A solo developer could plan roughly eight weeks for the staged scope, but actual duration depends on hardware, model downloads, model integration and dataset preparation. Treat the phase exit criteria as the source of truth; do not optimize for calendar dates at the expense of correctness.

10.2 Global definition of done

- Code is integrated into the existing repository without deleting unrelated working features.
- All relevant unit/integration tests pass; any failing test is explained, not hidden or skipped without reason.
- New endpoints have request/response schemas and error behavior documented.
- Configuration and secrets are not hard-coded.
- Logs provide actionable diagnostics without exposing sensitive media or credentials.
- README setup instructions work in a clean environment.
- The phase's acceptance criteria are demonstrated through tests or an explicit smoke-test transcript.

11. Testing and model evaluation

11.1 Software tests

Test level	What to test	Examples
Unit	Pure functions and small components.	Timestamp conversion, filename sanitization, schema validation, constraint evaluator, score normalization.
Adapter tests	Model/storage interfaces independent from external heavyweight dependencies.	Mock encoder output has correct dimension; storage refuses path traversal; model load errors are actionable.
Integration	API + DB + worker/storage using test dependencies.	Upload -> queue -> ready; failed job -> retry; delete cleans rows and files.
End-to-end smoke	Core user flow through API/UI.	Upload fixture -> wait ready -> search known concept -> click result -> correct playback timestamp.
Security/negative	Unauthorized access and invalid input.	User A cannot list/search/read/delete User B media; unsupported media rejected; malformed query bounded.
Regression benchmark	Retrieval behavior on a fixed query/video set.	Recall@K and localization metrics compared against baseline and previous saved run.

11.2 Benchmark design

Start with approximately five short development videos and around 20 queries for the initial proof of concept. Then create a larger held-out benchmark for meaningful evaluation. The initial set is for smoke testing and failure discovery, not a statistically strong performance claim.

Annotate at least: query text, video ID, relevant start/end time, query type, relevant result IDs (if multiple), and whether no matching event exists. Keep development and test clips or scenes separated to reduce leakage when fine-tuning or tuning thresholds.

11.3 Query categories

- Object/scene: “Find a blue car beside a building.”
- Attribute: “Find the person wearing a red shirt.”
- Single action: “Find someone opening a door.”
- Interaction: “Find someone passing a book to another person.”
- Ordered actions: “Find a person waving before sitting down.”
- Temporal extremum: “Find the first time the dog appears.”
- Audio-visual: “Find where the audience begins laughing,” only if audio is indexed.
- Hard negative / no match: similar scene but wrong action, wrong role, reversed order, or absent event.

11.4 Core metrics

Metric	Use	Notes
Recall@K	Whether at least one relevant result appears in top K.	Define relevance carefully; report K values such as 1, 5, 10.
Temporal IoU (tIoU)	Overlap between predicted and ground-truth time intervals.	Report thresholds such as 0.3 and 0.5 if suited to the task.
MRR / nDCG	Ranking quality when multiple results have graded relevance.	Use only if benchmark relevance labels support them.
Latency	User-facing search time, with percentile reporting.	Measure vector retrieval separately from expensive reranking.
Indexing throughput	Processing time per video minute or media hour.	Report hardware, frame interval, model, and video resolution.
Storage / vector count	Derived storage per video hour.	Include thumbnails and database overhead separately when possible.
Calibration / OOD metrics	Optional for future uncertainty work.	Do not call raw similarity a calibrated probability.

11.5 Research experiment

Research hypothesis: incorporating temporal features and structured event-role/ordering information improves retrieval of multi-step natural-language queries compared with frame-level semantic search.

Compare three systems on the same held-out queries: A) frame embeddings; B) video-window embeddings; C) candidate event descriptions plus structured temporal constraints. Report improvements and regressions by query type, with indexing cost and latency. Do not claim the hypothesis is validated unless results support it.

11.6 Fine-tuning gate

Do not begin fine-tuning until: (1) the baseline works, (2) a labeled benchmark exists, (3) hard negatives are documented, (4) train/validation/test splits are fixed, and (5) evaluation can be run reproducibly. A possible ranking loss uses a positive query-video pair and a hard negative, but the objective, data, and implementation should be documented and compared against the pretrained baseline.

12. Privacy, security, reliability, and operations

12.1 Media privacy and access control

- Treat uploaded videos, thumbnails, extracted frames, transcripts, embeddings, captions, and event records as private user data.
- Require authentication in multi-user mode. For local MVP, still structure APIs and database queries around owner_id to avoid redesign later.
- Enforce authorization at every read, search, stream, and delete endpoint.
- Use opaque storage keys; never derive filesystem paths directly from an uploaded filename.
- Do not publish object-storage URLs or media directories openly by default.
- Support full deletion of original media and derived artifacts. Document any backups or retention exceptions.

- Do not send private videos to third-party APIs without explicit configuration, disclosure, and user consent.

12.2 Upload and file-processing security

- Validate file size, extension, MIME indication, and actual media streams using FFprobe.
- Run FFmpeg with safe argument lists (no shell string interpolation from user input).
- Use a generated file key and write into a controlled directory; prevent path traversal and symlink escapes.
- Set timeouts/resource limits for malformed media, decode loops, and extremely long videos.
- Avoid exposing raw FFmpeg errors to end users; log detailed diagnostics server-side.
- Use rate limits and per-user storage/processing quotas in any shared deployment.

12.3 Reliability

- Jobs need retry count, stage, and terminal failure state. Retry only transient failures automatically.
- Persist job state so worker restart does not lose the only record of ongoing work.
- Use idempotency keys or unique extraction identities for safe retries.
- Keep readiness checks separate from liveness checks; a healthy API process is not necessarily ready to search if the database is down.
- Include a manual re-index action in developer/admin workflow so model or sampling changes can rebuild derived data.
- Do not mix embeddings generated by different model versions in one index without model-aware filtering.

12.4 Observability

Log structured fields such as request_id, video_id, job_id, stage, model_id, elapsed_ms, frames_processed, and error_code. Monitor upload failures, processing time, queue depth, embedding throughput, search latency, empty-result rate, and user-reported false matches. Do not log video content, authorization tokens, API keys, or full private transcripts unnecessarily.

13. Agent execution protocol (Codex / Antigravity)

13.1 Required behavior before coding

7. Inspect the repository, existing framework, package manager, scripts, and tests before editing anything.
8. Summarize the current state: existing functionality, missing components, likely blockers, and files to modify.
9. Use the existing stack and conventions where possible. Do not replace a working architecture merely because this document suggests a different layout.
10. Create a short task checklist mapped to phase IDs and requirement IDs.
11. Identify any required secrets, model downloads, system binaries, or GPU dependencies; provide safe mock mode if practical.
12. If a detail is not essential to proceed, choose a reasonable documented default rather than repeatedly asking for clarification.

13.2 Required behavior while coding

13. Implement the smallest end-to-end vertical slice, then add the next capability.
14. Keep changes modular. Use service/model/storage abstractions instead of importing model libraries throughout route handlers.
15. Write or update tests with each meaningful feature. Run the tests after editing, not just at the end.
16. Use database migrations and update .env.example and README when configuration changes.
17. Never hard-code real credentials, fabricated model outputs, fake confidence values, or success responses that bypass actual processing.
18. Do not silently swallow exceptions. Convert expected failures to safe typed errors and log useful diagnostic context.
19. Do not skip model license checks or dataset-license review when adding a model/dataset.
20. If an advanced model is unavailable, implement an explicit adapter boundary and a clear feature-disabled message; do not fake its results.

13.3 Required status report after each phase

```
Phase: <phase ID and name>
Implemented:
- <observable changes>
Files changed:
- <key files>
Commands run:
- <tests / lint / build / smoke test>
Results:
- <pass/fail plus relevant output>
Acceptance criteria:
- [x] <criterion passed>
- [ ] <criterion not yet met, with reason>
Known limitations / next risks:
- <specific notes>
Next phase proposal:
- <one focused increment>
```

13.4 Copy-ready agent kickoff prompt

PROMPT FOR THE CODING AGENT

You are implementing Memory Compiler using the attached technical specification. First inspect the current repository and report its existing structure, stack, runnable commands, tests, and working functionality. Do not overwrite working code blindly. Then propose a short plan mapped to the specification's requirement IDs and phases. Implement Phase 0 and Phase 1 only unless those are already complete. Keep changes small and testable. Provide a deterministic mock embedding provider for tests and avoid downloading large model weights in CI. Add migrations, configuration, tests, and README updates with the implementation. Run tests and report exact commands and results. Do not claim a feature is implemented unless it runs end to end. After each phase, list changed files, acceptance criteria passed/blocked, limitations, and the next phase. Continue phase by phase, prioritizing a working upload-to-search vertical slice over advanced models.

14. Risks, open decisions, and non-goals

14.1 Main risks

Risk	Impact	Mitigation / decision
Sparse frame sampling	Brief actions are missed entirely.	Keep interval configurable; add temporal-window or adaptive sampling; measure recall versus cost.
Semantic similarity confuses actions	Pickup, set-down, and handover may look similar.	Use hard negatives; video encoder; explicit event/role/temporal constraints.
Model captions hallucinate	False event claims may surface in results.	Treat descriptions as hypotheses; retain evidence and provenance; rerank/verify top candidates.
Track ID switches	Participant role matching becomes unreliable.	Track within scenes initially; test ID switches; avoid cross-scene identity claims.
Large model dependencies	Setup and CI become brittle.	Pin dependencies; mock adapters in CI; document weight downloads and device requirements.
Long processing jobs	API timeouts and memory pressure.	Queue workers, batching, checkpoints, resource limits, retry-safe stages.
Sensitive video exposure	Privacy breach or unauthorized access.	Private storage, ownership checks, deletion, cautious third-party processing policy.

Risk	Impact	Mitigation / decision
Model upgrades change vectors	Existing index becomes incompatible.	Version embeddings; require explicit re-index or parallel indexes when switching models.

14.2 Decisions that should remain configurable

- Frame sample interval and maximum frames per video.
- Image-text encoder ID, revision, and preprocessing parameters.
- Embedding batch size and device.
- Top-K candidates and deduplication/temporal merge tolerance.
- Search score thresholds and any score combination weights.
- Maximum upload size, allowed file types, and job timeout.
- Optional model features (scene detection, video encoder, audio transcription, object tracking).

14.3 Keep out of the first release

Do not add user-to-user sharing, social features, persistent cross-video identity tracking, face search, automated inference about emotions or intent, a graph database, or custom foundation-model training unless a validated requirement emerges. Each adds cost, privacy implications, complexity, or evaluation burden without being necessary for the MVP.

15. Reference links and terminology

Use official documentation and original research sources where possible. Verify the current package/API version and license at implementation time; these links are starting points, not a substitute for dependency and license review.

15.1 Implementation references

FastAPI background tasks — Guidance on request-background work and when a larger queue such as Celery is appropriate.

PySceneDetect documentation — Scene-boundary detection and Python API.

pgvector — Vector columns, similarity operators, exact and approximate search in PostgreSQL.

FAISS documentation — Alternative vector search/indexing toolkit; introduce only if justified by measurements.

OpenCLIP — Open implementation and model ecosystem for CLIP-style text/image embeddings; inspect checkpoint licenses.

Hugging Face video classification docs — Starting point for pretrained video classification models and inference workflows.

FFmpeg / FFprobe — Probe media metadata and streams.

Celery documentation — Distributed task queue and worker architecture.

ByteTrack — Candidate multi-object tracking implementation; Phase 3+ only.

OpenAI Whisper repository — Optional speech transcription component; inspect deployment and license needs.

ActivityNet Captions repository — Dataset/research resource for dense video descriptions; verify access and terms.

Charades-STA related implementation — Research code associated with temporal activity localization; verify dataset/model compatibility.

15.2 Terminology

Term	Meaning in this project
Embedding	A numeric vector produced by a model to represent content for similarity search.
Frame-level retrieval	Searching individual sampled frames; useful for visual concepts but weak evidence for actions.
Video-window embedding	A vector representing a short frame sequence, potentially encoding temporal information.

Term	Meaning in this project
Temporal localization	Predicting the start and end timestamps of a query-relevant event.
tIoU	Temporal intersection over union between a predicted and ground-truth interval.
Hard negative	A non-matching example that looks semantically similar to the correct example.
Provenance	Metadata documenting how an event description or label was produced and what evidence supports it.
Idempotency	The ability to retry an operation without producing duplicate or inconsistent state.
Out-of-distribution input	An input that differs materially from the data on which a model was trained or validated.

Final build directive

SHIP THE EVIDENCE-BACKED VERTICAL SLICE FIRST

The first successful release is: upload a video -> process timestamped frames -> index compatible image-text embeddings -> search with a natural-language query -> return a candidate timestamp -> play the original video at that time. Only after this path is tested and benchmarked should the agent add video-level action recognition, event graphs, temporal constraints, audio features, or fine-tuning.

End of specification.